

CLASSIFICATION WITH MULTILAYER PERCEPTRON (MLP)

BREAST CANCER BINARY CLASSIFICATION USING BACKPROPAGATION



Rawan Faisal Alotaibi

Submitted to: Dr. Huda Hakamii

1. Introduction

In this assignment, a Multilayer Perceptron (MLP) neural network was implemented from scratch to solve a binary classification problem using the Wisconsin Diagnostic Breast Cancer dataset. The main objective of this work is to classify tumors as either benign or malignant based on thirty numerical features extracted from medical images.

The implementation includes manual forward propagation, backpropagation, and squared-loss error computation using Python and NumPy without relying on high-level machine learning or deep learning libraries. In addition, different learning rates and hidden layer sizes were evaluated to analyze their impact on the classification performance of the network.

2. Dataset

The dataset used is wdbc.data, which comes from the UCI Machine Learning Repository. It consists of 569 instances.

Each record contains a unique patient identification number, a diagnosis label where M stands for malignant and B stands for benign, and thirty real-valued features representing cell nuclei characteristics like radius, texture, perimeter, and area.

The properties of the dataset and the parameters used for evaluation are summarized below:

Property	Value
Total instances	569
Number of features	30
Class labels	Benign (B $\rightarrow$ 0), Malignant (M $\rightarrow$ 1)
Training set size	80% (455 instances)
Test set size	20% (114 instances)
Data Splitting	Stratified split using scikit-learn
Feature Scaling	Manual standardization with zero mean and unit variance

3. MLP Architecture

We designed a fully connected three-layer neural network for this classification task. This structure is efficient enough to learn the non-linear boundaries needed to separate the data samples.

3.1 Network Design

The network consists of three main layers:

Input Layer:

Contains 30 neurons, where each neuron corresponds to one feature from the dataset.

Hidden Layer:

Contains 15 neurons that use the Sigmoid activation function.

Output Layer:

Contains a single neuron that also uses the Sigmoid activation function to produce a final classification probability between 0 and 1.

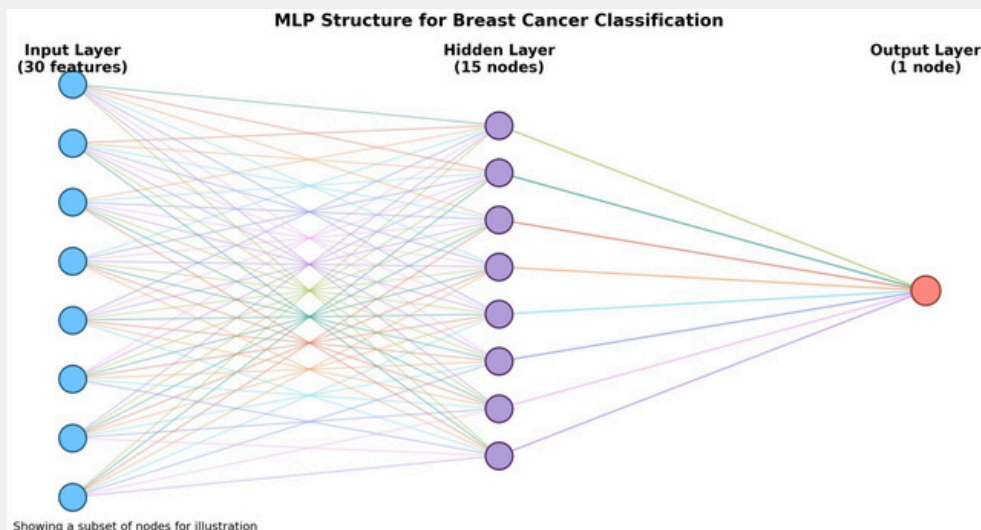


Figure 1: MLP Structure for Breast Cancer Classification

3.2 Activation Function

The sigmoid function is used in both layers: $\sigma(z) = 1 / (1 + e^{(-z)})$

Its derivative, used in backpropagation, is: $\sigma'(z) = a \times (1 - a)$

The sigmoid function maps any real value into the range (0, 1), which makes it suitable for binary classification.

3.3 Weight Initialization

Weights were initialized using a normal distribution with mean 0 and standard deviation 0.01. This small-value initialization helps prevent early saturation of the sigmoid activation function during training. Biases were initialized to zero.

4. Training and Backpropagation

The network was trained using mini-batch gradient descent with a batch size of 32. The weights were updated using the backpropagation algorithm combined with a squared-loss function, which measures the mean squared error between predictions and true labels.

4.1 Loss Function

The squared-loss is calculated using the formula :

$$L = (1/m) \times \sum (\hat{y} - y)^2$$

Where $\hat{y}$ represents the predicted output of the network, y is the actual true label, and m is the total number of samples in the current batch.

4.2 Backpropagation Algorithm

Each training epoch goes through the following steps:

- Shuffle the training data and split into mini-batches of size 32
- Forward pass: compute activations through all layers
- Compute the squared loss between predictions and true labels
- Backward pass: compute gradients via the chain rule
- Update the network weights and biases: $W = W - \eta \times \partial L / \partial W$

4.3 Training Results

The model was trained for 100 epochs using a learning rate of 0.5 and 15 hidden nodes. The figure below shows how the training and test loss decrease over time, alongside the corresponding accuracy curves.

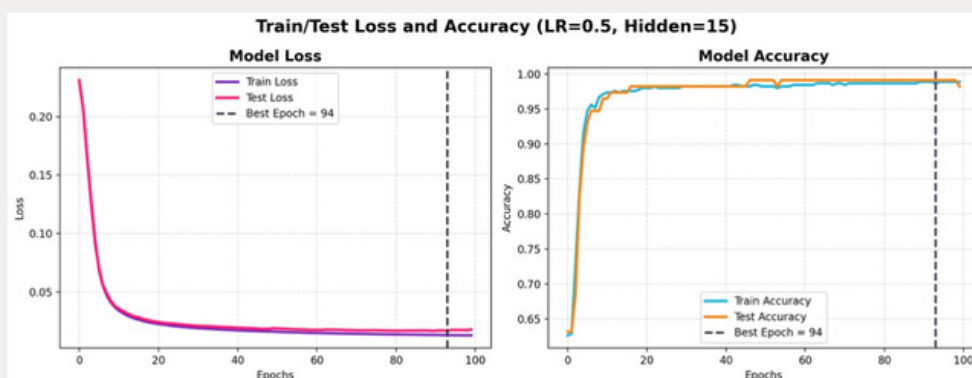


Figure 2: Train/Test Loss and Accuracy (LR=0.5, Hidden=15)

The loss on both the training and test sets decreases sharply during the first 20 epochs, then continues to decrease gradually. The accuracy curves show a similar trend, rising quickly to approximately 98–99% before stabilizing. The training and test curves remain very close throughout the training process, indicating that the model does not suffer from significant overfitting.

4.4 Ideal Number of Iterations

Based on Figure 2, the best checkpoint was recorded at epoch 94, where the test loss reached its minimum value. The model converged successfully within 100 epochs, and additional training may provide only minor improvements. Therefore, epoch 94 can be considered the optimal stopping point for this configuration.

5. Model Evaluation

5.1 Evaluation Metrics

In addition to accuracy, the model was evaluated using Precision, Recall, and F1-Score to provide a more comprehensive assessment of the classification performance. These evaluation metrics are particularly important in medical classification tasks, where class imbalance may make accuracy alone insufficient for measuring model performance.

Accuracy	Precision	Recall	F1-Score
98.2%	100.0%	95.2%	97.6%

5.2 Discussion

The model achieved an overall accuracy of 98.2% on the test set. Precision reached 100%, meaning that every case classified as malignant by the model was truly malignant, with no false positives. Recall was 95.2%, indicating that the model correctly identified 95.2% of all actual malignant cases while missing only a very small number of them.

In medical classification tasks such as cancer detection, Recall is considered one of the most important evaluation metrics because missing a malignant case (false negative) can be far more dangerous than producing an unnecessary follow-up (false positive). The model performed very well in this regard, and the F1-Score of 97.6% reflects a strong balance between Precision and Recall.

6. Effect of Learning Rate

6.1 Purpose of the Learning Rate

The learning rate (η) is a hyperparameter that controls the step size during weight updates. The update rule is:

$$W_{\text{new}} = W_{\text{old}} - \eta \times \partial L / \partial W$$

If η is too large, weight updates may overshoot the minimum and training can become unstable. If it is too small, convergence becomes very slow and many more epochs are required to achieve acceptable performance. Therefore, choosing an appropriate learning rate is one of the most important factors when training a neural network.

6.2 Comparison of Learning Rates

The same MLP architecture was trained using four different learning rates: [1.0, 0.5, 0.1, 0.01], while keeping all other settings constant (100 epochs and 15 hidden nodes). The results are shown in Figure 3.

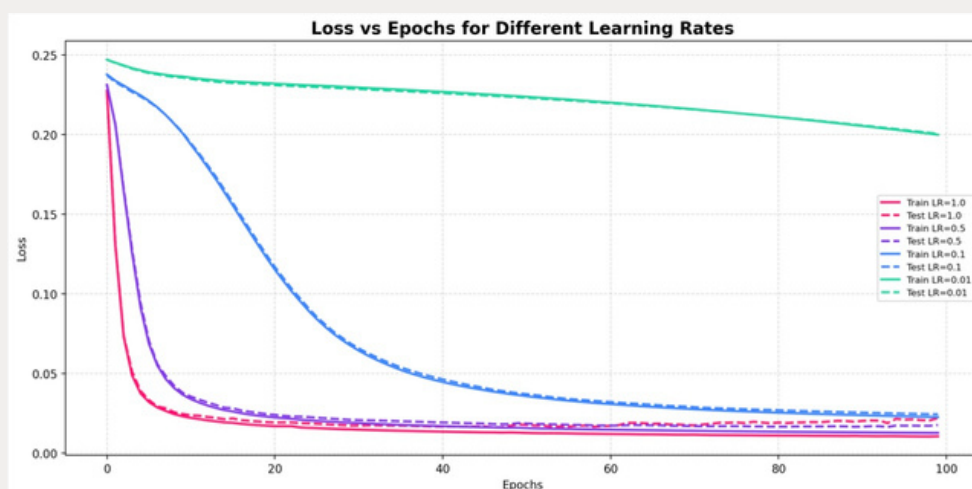


Figure 3: Loss vs Epochs for Different Learning Rates

- LR = 1.0 (pink): Fastest convergence. The loss decreases sharply during the first few epochs and reaches a very low value near the end of training.
- LR = 0.5 (purple): Second fastest convergence. The performance is very close to LR = 1.0, but with slightly smoother and more stable training behavior.
- LR = 0.1 (blue): Noticeably slower convergence. The loss continues decreasing gradually even near epoch 100, suggesting that additional epochs may still improve performance.
- LR = 0.01 (green): Very slow convergence. The loss decreases very slowly over 100 epochs and remains relatively high compared to the other learning rates. This learning rate is too small for efficient convergence on this dataset.

For this dataset, LR = 0.5 provides the best balance between convergence speed and training stability. Although LR = 1.0 converges slightly faster, LR = 0.5 shows smoother convergence while achieving comparable final performance.

7. Effect of Hidden Layer Size

7.1 Experiment Setup

We varied the number of hidden nodes (k) across [5, 10, 15, 20, 25, 30], while keeping all other hyperparameters fixed (learning rate = 0.5 and epochs = 100). For each value of k , the model was trained using five different random seeds, and the final test accuracy was averaged to provide a more reliable comparison between configurations.

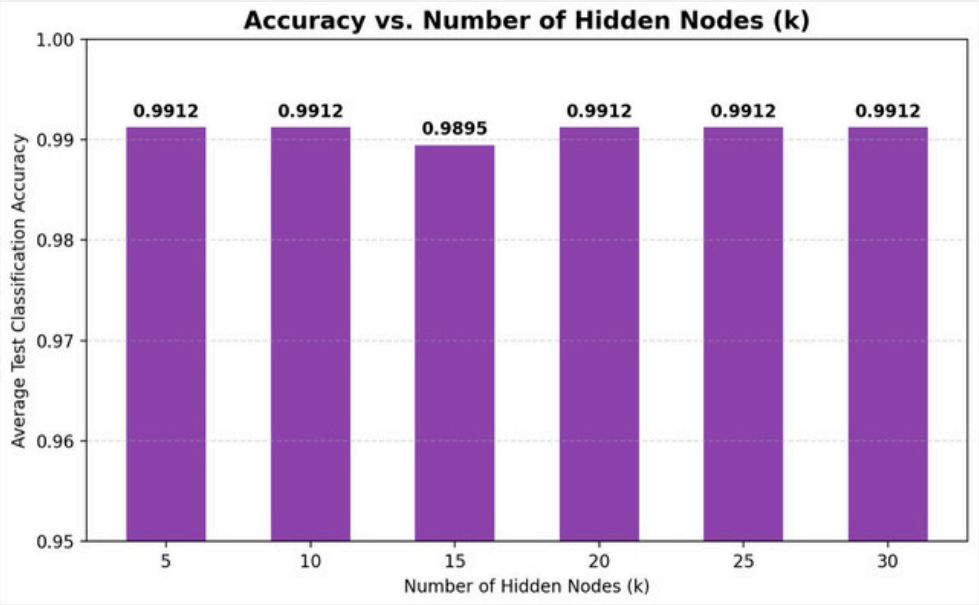


Figure 4: Accuracy vs. Number of Hidden Nodes (k)

7.2 Analysis and Trends

As shown in Figure 4, most hidden layer sizes achieved an average test accuracy of approximately 99.1%. The configuration with $k = 15$ achieved a slightly lower accuracy (98.95%), while the remaining configurations produced nearly identical results.

These results indicate that the WDBC dataset can be classified effectively even with relatively small hidden layers. Increasing the number of hidden nodes beyond this range does not provide a significant improvement in classification performance.

Based on these observations, $k = 15$ was selected as the base configuration because it provides strong performance while maintaining a relatively simple network architecture. Larger hidden layers increase computational complexity without producing noticeable gains in accuracy for this dataset.

8. Summary of Findings

Aspect	Finding
Best stopping epoch	Epoch 94 (lowest test loss)
Best learning rate	LR = 0.5 (best balance between speed and stability)
Best hidden layer size	k = 15 (good balance between accuracy and complexity)
Final test accuracy	98.2% (LR=0.5, k=15, 100 epochs)
Best accuracy (all k)	99.12% (achieved by k = 5, 10, 20, 25, and 30)
Precision	100.0% , no false positives
Recall	95.2% , critical for medical diagnosis
F1-Score	97.6% , strong balance of precision and recall
Overfitting observed	No significant overfitting observed

9. Conclusion

In this assignment, we built and trained an MLP from scratch using only NumPy, without relying on any machine learning frameworks, to classify breast cancer cases as benign or malignant. The network was trained using the squared-loss function and backpropagation with mini-batch gradient descent.

The results showed that the model achieved a test accuracy of 98.2%, with a precision of 100%, recall of 95.2%, and an F1-Score of 97.6%. The lowest test loss was recorded at epoch 94. Among the learning rates tested, LR = 0.5 provided the best balance between convergence speed and training stability.

Regarding hidden layer size, most tested values of k from 5 to 30 achieved an average test accuracy of approximately 99.1%, while k = 15 achieved a slightly lower accuracy of 98.95%. These results indicate that relatively small networks are sufficient for learning the patterns in this dataset effectively.

Overall, the MLP implemented in this assignment demonstrated strong generalization performance, no significant overfitting, and promising performance for breast cancer diagnosis support.